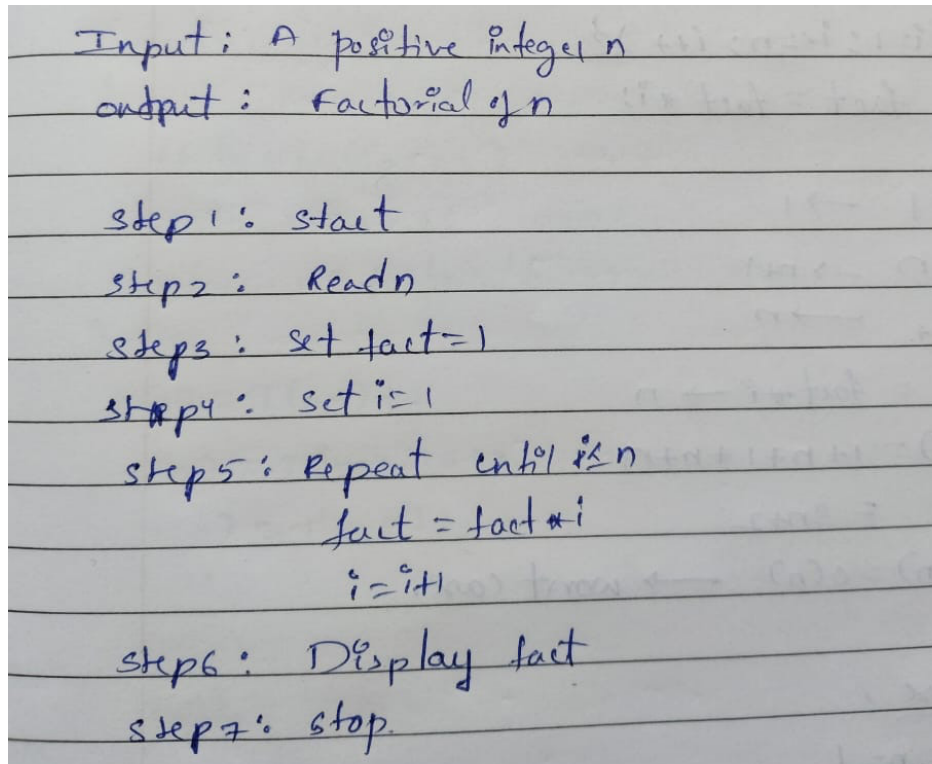


Practical- 4

Aim: Implementation and Time analysis of factorial program using iterative and recursive method.

Iterative Method

Algorithm:



Input: A positive integer n
output: Factorial of n

step 1: start
step 2: Read n
step 3: set fact = 1
step 4: set i = 1
step 5: Repeat until i ≤ n
 fact = fact * i
 i = i + 1
step 6: Display fact
step 7: stop

Program:

```
#include <iostream>
using namespace std;
class Factorial {
public: int fact(int n)
{ int result = 1;

    for (int i = 1; i <= n; i++)
        { result = result * i; }

    return result;
}
};

int main()
```

```
{
    int n,result;
    Factorial obj;

    cout << "Enter the value to find its factorial: ";
    cin >> n;

    result = obj.fact(n);

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:

```
● lokeshgandreti@LOKESHs-MacBook-Air practical-3 % cd "/Users/lokeshgandreti/Desktop/c++/practical - 4/" && g++ fa
ctIterative.cpp -o factIterative && "/Users/lokeshgandreti/Desktop/c++/practical - 4/"factIterative
Enter a number: 5
Factorial of 5 = 120
○ lokeshgandreti@LOKESHs-MacBook-Air practical - 4 %
```

Time Complexity:

for ($i=1; i \leq n; i++$)
 $fact = fact * i$
}

$i=1 \rightarrow 1$
 $i \leq n \rightarrow n+1$
 $i++ \rightarrow n$
 $fact = fact * i \rightarrow n$
 $T(n) = 1 + n + 1 + n + n$
 $= 3n + 2$
 $T(n) = O(n) \rightarrow$ worst case.

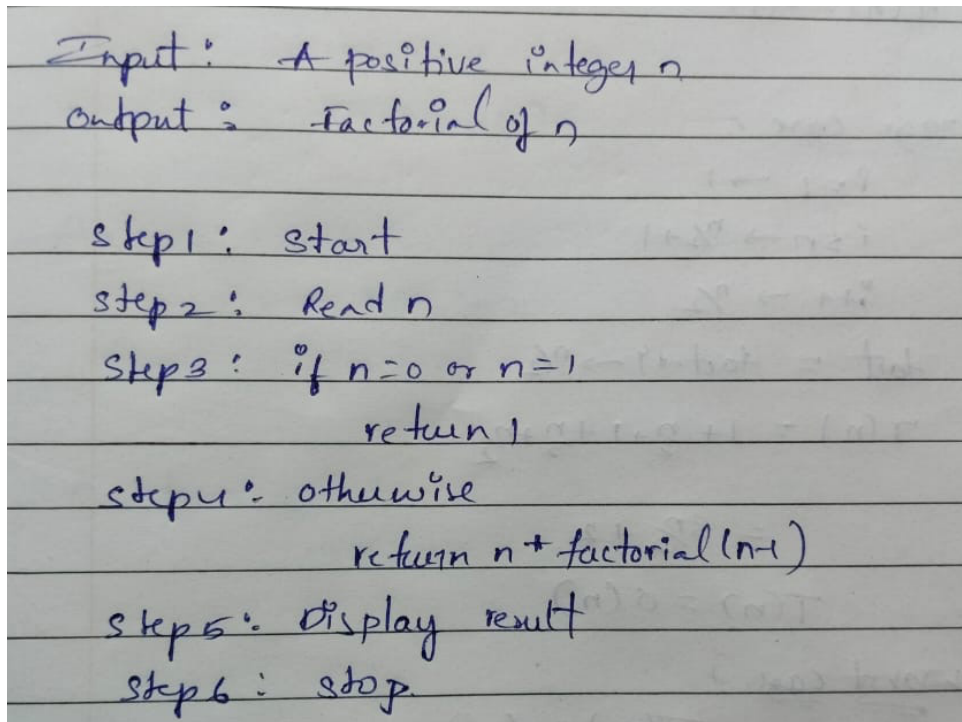
Best case
if $n=1$
 $T(n) = 1 + 2 + 1 + 1$
 $= 5$
 $T(n) = O(1)$

Average case
 $i=1 \rightarrow 1$
 $i \leq n \rightarrow \frac{n}{2} + 1$
 $i++ \rightarrow \frac{n}{2}$
 $fact = fact * i \rightarrow \frac{n}{2}$
 $T(n) = 1 + \frac{n}{2} + 1 + \frac{n}{2} + \frac{n}{2}$
 $= \frac{3n}{2} + 2$
 $T(n) = O(n)$

Worst case
 $T(n) = O(n)$

Recursive Method

Algorithm:



Input: A positive integer n
Output: Factorial of n

Step 1: Start
Step 2: Read n
Step 3: if $n=0$ or $n=1$
 return 1
Step 4: otherwise
 return $n * \text{factorial}(n-1)$
Step 5: Display result
Step 6: stop

Program:

```
#include <iostream>
using namespace std;

class Factorial { public:
int factr(int n) { if (n ==
0 || n == 1) { return 1;

}
else
{
int result = n * factr(n - 1);
return result;
}
}
};

int main()
{
int n, result;
Factorial obj;

Name: Lokesh.G
Enrollment Number:92460118137
```

```
cout << "Enter the value to find its factorial: ";
cin >> n;

result = obj.factr(n);

cout << "Factorial of " << n << " is " << result << endl;

return 0;
}
```

Output:

```
lokeshgandreti@LOKESHs-MacBook-Air practical - 4 % cd "/Users/lokeshgandreti/Desktop/c++/practical - 4/" && g++ factreccursive.cpp -o factreccursive && "/Users/lokeshgandreti/Desktop/c++/practical - 4/"factreccursive
Enter a number: 6
Factorial of 6 = 720
lokeshgandreti@LOKESHs-MacBook-Air practical - 4 %
```

Time complexity:

factorial(n)

```

{
    if (n == 0 || n == 1) → 1
        return 1; → 1
    return n * factorial(n-1); → T(n-1)
}

```

$T(n) = T(n-1) + 2$
 $T(n) = T(n-2) + 2 + 2$
 $T(n) = T(n-3) + 2 + 2 + 2$
 $\vdots$
 $T(n) = T(1) + 2n$
 $T(n) = O(n)$

Best Case :
 when $n = 0$ or $n = 1$:
 $T(n) = 1$
 $T(n) = O(1)$

Average case :
 $T(n) = T(n-1) + 2$
 $T(n) = O(n)$

Worst case :
 $T(n) = O(n)$



Conclusion:

The factorial program was implemented using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.